Introduction

This laboratory work explores multi-task learning on the MNIST dataset, where the model predicts, the digit (0–9), its parity (odd/even) and threshold classification (<5 or ≥5).
The primary objective was to improve model accuracy and generalization by experimenting with different CNN architectures, activation functions, normalization layers, dropout regularization, and learning rate schedulers.

To improve modularity, reproducibility, and training efficiency, I refactored the original notebook using PyTorch Lightning, to automate device management, callbacks (checkpoints, early stopping) and cleaner logging.
All critical parts of the model were commented for clarity. Confusion matrices for each task (digit, parity, threshold) were generated and saved for inclusion in the final submission.

Task

1- Modify the original CNN architecture and training setup improve classification accuracy.
2- Experiment with different model components and hyperparameters:
    a. Convolutional Layers, and filters
    b. Normalization and dropout
    c. Activation functions
    d. Optimizers and learning rate schedulers
3- Compare results across experiments using training, validation, test metrics

Code

Full Implementation of the model:

- Model definition: Multi-output CNN with shared convolutional backbone and three heads. (digit, parity, threshold)
- PyTorch Lightning module: Handles training, validation, and optimization automatically.
- Training setup: Includes ModelCheckpoints, EarlyStopping functions, and different scheduler, optimizer, activation functions, and utility functions.

Architecture and Lightning Wrapper for the model:

I removed the comments just to fit the picture better, but in the notebook they are present.

```
class DigitParityCNN(nn.Module):
    def __init__(self):
        super(DigitParityCNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)

        self.fc1 = nn.Linear(128 * 3 * 3, 256)
        self.fc2_digit = nn.Linear(256, 10)  # Digit classification (0-9)
        self.fc2_parity = nn.Linear(256, 2)  # Parity classification (odd/even)
        self.fc2_threshold = nn.Linear(256, 2)  # Threshold classification (<5 or

        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()
        self.flatten = nn.Flatten()

        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(128)
        self.bn3 = nn.BatchNorm2d(256)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x) # normalize before relu
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv2(x)
        x = self.bn2(x) # normalize before relu
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv3(x)
        x = self.bn3(x) # normalize before relu
        x = self.relu(x)
        x = self.pool(x)

        x = self.flatten(x)

        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)   # randomly drop some neurons, helps with overfitting

        digit_output = self.fc2_digit(x)
        parity_output = self.fc2_parity(x)
        threshold_output = self.fc2_threshold(x)

        return digit_output, parity_output, threshold_output
```

```
ss DigitParityLightning(pl.LightningModule):
def __init__(self, learning_rate=1e-3, epocs = 10):
    super(DigitParityLightning, self).__init__()
    self.save_hyperparameters()
    self.model = DigitParityCNN()
    self.loss_fn = nn.CrossEntropyLoss()

    self.acc_digit = torchmetrics.Accuracy(task="multiclass", num_classes=10)
    self.acc_parity = torchmetrics.Accuracy(task="multiclass", num_classes=2)
    self.acc_threshold = torchmetrics.Accuracy(task="multiclass", num_classes=2)

    self.train_losses, self.val_losses = [], []
    self.train_digit_acc, self.val_digit_acc = [], []
    self.train_parity_acc, self.val_parity_acc = [], []
    self.train_threshold_acc, self.val_threshold_acc = [], []

def forward(self, x): # Just using the forward method of the model nothing to change here
    return self.model(x)
def _common_step(self, batch, stage):
    images, (labels, parities, thresholds) = batch
    digit_out, parity_out, thresh_out = self(images)

    loss_digit = self.loss_fn(digit_out, labels)
    loss_parity = self.loss_fn(parity_out, parities)
    loss_thresh = self.loss_fn(thresh_out, thresholds)
    loss = loss_digit + loss_parity + loss_thresh

    acc_d = self.acc_digit(digit_out, labels)
    acc_p = self.acc_parity(parity_out, parities)
    acc_t = self.acc_threshold(thresh_out, thresholds)

    self.log(f"{stage}_loss", loss, prog_bar=True)
    self.log(f"{stage}_digit_acc", acc_d, prog_bar=True)
    self.log(f"{stage}_parity_acc", acc_p, prog_bar=True)
    self.log(f"{stage}_threshold_acc", acc_t, prog_bar=True)
    return loss

def training_step(self, batch, batch_idx): # Just calls the _common_step
    return self._common_step(batch, "train")
def validation_step(self, batch, batch_idx):
    return self._common_step(batch, "val")

def configure_optimizers(self):

    optimizer = torch.optim.AdamW(self.parameters(), lr=self.hparams.learning_rate, weight_decay=
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.9)
    return {"optimizer": optimizer, "lr_scheduler": scheduler}

def on_train_epoch_end(self):
    self.train_losses.append(self.trainer.callback_metrics['train_loss'].item())
    self.train_digit_acc.append(self.trainer.callback_metrics['train_digit_acc'].item())
    self.train_parity_acc.append(self.trainer.callback_metrics['train_parity_acc'].item())
    self.train_threshold_acc.append(self.trainer.callback_metrics['train_threshold_acc'].item())
def on_validation_epoch_end(self):
    self.val_losses.append(self.trainer.callback_metrics['val_loss'].item())
    self.val_digit_acc.append(self.trainer.callback_metrics['val_digit_acc'].item())
    self.val_parity_acc.append(self.trainer.callback_metrics['val_parity_acc'].item())
    self.val_threshold_acc.append(self.trainer.callback_metrics['val_threshold_acc'].item())
```

Result

For all the experiments, epochs = 10, learning rate = 0.001, batch size = 64.

Experiment	Changes	Scheduler/Activation	Test Loss	Digit Accuracy (%)	Parity Accuracy (%)	Threshold Accuracy (%)
0	Base Model	StepLR(step_size = 1, gamma = 0.5) Adam	0.0518	99.24	99.56	99.46
1 (Ex1)	Batch normalization before ReLu, Dropout(0.4), 10% extra validation.	StepLR(step_size = 3, gamma = 0.9), AdamW(weight_decay = 0.02)	0.0518	99.40	99.72	99.47
2	Ex1 LeakyReLu(0.2) instead of ReLu	Ex1	0.0763	99.06	99.53	99.39
3	Ex1 + Extra Conv layer (128, 256)	Ex1	0.0510	99.46	99.72	99.44
4	Ex1	CosineAnnealing(epochs)	0.0556	99.46	99.72	99.44
5	Ex1	ReduceLROnPlateau()	0.0726	99.09	99.62	99.31
6	Ex1 with Double CNN Neurons (64 -> 128->256)	Ex1	0.0563	99.17	99.50	99.41

Overall summary:

- Normalization before ReLu and Dropout after Full Connected layer improved convergence and stability.
- LeakyReLu did not benefit over ReLu.
- Extra Convolution layer showed pretty good results but since the dataset is fairly small, I did not want to risk overfitting.
- StepLR scheduler gave the most balanced, and better results compared to other schedulers.
- Adaptive Average Pooling gave incredibly bad results where loss is double in most cases, so I did not included it in the table.
- Batch normalization and dropout functionality proved to be the best improvement I was able to apply.

I have included the best models, confusion matrices and training graph in the submission.

Conclusion

Through iterative experimentation, the best-performing configuration was a 3-layer CNN (32→64→128) with batch normalization before ReLU, dropout (0.4) after the first fully connected layer, and training using the AdamW optimizer with a StepLR scheduler ($\gamma = 0.9$, step_size = 3).

Although the 4-layer CNN variant (32→64→128→256) occasionally achieved slightly better results, its improvements were inconsistent across runs. Moreover, the increased training time and computational cost were not justified by the marginal and unstable performance gains. The 3-layer architecture consistently demonstrated strong and reliable results, making it the more practical choice.

Best metrics achieved:

- Test Loss: 0.0518
- Digit Accuracy: 99.39%
- Parity Accuracy: 99.72%
- Threshold Accuracy: 99.47%

Finally, techniques like evolutionary search or automated hyperparameter optimization (Optuna) could be used to search for optimal combinations of batch size, learning rate, optimizer, scheduler, and network depth (number of layers, and neurons).